ITAQUERIAERO — SISTEMA DE SUPORTE À DECISÃO METEOROLÓGICA

Relatório Técnico de Validação e Testes do Sistema Web (Padrão UNIVESP)

4. METODOLOGIA E EXECUÇÃO DOS TESTES DO SISTEMA WEB

Para validar a confiabilidade e o comportamento dinâmico da interface web do sistema **ItaqueriAero**, a equipe de desenvolvimento projetou e realizou um plano de testes funcionais do tipo caixa-preta. O objetivo principal foi garantir que a aplicação web, desenvolvida sob o framework **Django**, fosse capaz de interpretar e renderizar corretamente as três classes de risco meteorológico estabelecidas pelas normativas operacionais da **CBVL** (Confederação Brasileira de Voo Livre), aplicadas diretamente à Rampa Leste da Serra do Itaqueri, em São Pedro/SP.

Com o intuito de assegurar o rigor científico exigido nos critérios de avaliação acadêmica da universidade e evitar terminantemente a utilização de dados fictícios ou simulados artificialmente, o processo experimental foi conduzido por meio de **Casos de Teste (identificados formalmente pela sigla CT)**. A estratégia metodológica dividiu-se em duas etapas operacionais bem delineadas: o isolamento laboratorial estático e a validação automatizada em tempo real (ambiente integrado).

4.1 Ambiente de Testes e Estratégia de Isolamento

A execução de toda a suíte de testes funcionais ocorreu em ambiente local utilizando o editor de código integrado **Visual Studio Code (VS Code)** sob o servidor de desenvolvimento embutido do Django (`localhost:8000`). Para viabilizar a análise individual de cada cenário meteorológico sem a interferência das oscilações instantâneas dos servidores externos de sensoriamento, a equipe realizou uma modificação técnica temporária na arquitetura do sistema.

O script de monitoramento contínuo em JavaScript (`monitor_clima.js`), localizado no componente de renderização do *frontend* (`dados_leitura.html`), foi temporariamente desativado por meio de marcações de comentários no arquivo de *template*. Esse procedimento permitiu isolar de forma estrita o fluxo de dados na camada de controle do backend (arquivo `views.py`), onde as variáveis ambientais de entrada foram amarradas estritamente com base nos dados meteorológicos reais do histórico do clube de voo livre (**CSPVL**).

4.2 Execução dos Cenários de Teste e Capturas de Tela

O sistema foi submetido a quatro execuções controladas, cujos resultados visuais foram integralmente documentados por meio de capturas de tela estruturadas para o corpo do trabalho escrito, conforme detalhado na tabela a seguir:

Código (ID)	Data Histórica	Vetor de Entrada (Dados Reais do Solo/Satélite)	Comportamento Esperado da Interface Web	Status
CT-01	01/01/2024	Pluviometria: 12.0 mm | Vento Médio: 7.7 km/h | Rajada: 45.4 km/h	Exibição do Alerta Crítico: **INSEGURO** (Tarja Vermelha)	**Aprovado**
CT-02	09/01/2024	Pluviometria: 0.0 mm | Vento Médio: 17.4 km/h | Rajada: 47.9 km/h	Exibição do Alerta de Transição: **ATENÇÃO** (Tarja Amarela)	**Aprovado**
CT-03	03/01/2024	Pluviometria: 0.0 mm | Vento Médio: 8.6 km/h | Rajada: 41.8 km/h	Exibição do Status Operacional: **SEGURO** (Tarja Verde)	**Aprovado**
CT-04	Tempo Real	Leitura Síncrona via API da Open-Meteo com Relógio do Sistema	Exibição Autônoma da Data Atual com Dados Dinâmicos dos Sensores	**Aprovado**

4.3 Análise Qualitativa dos Resultados

Os testes demonstraram integridade absoluta no fluxo de dados entre os dicionários de variáveis orientados a objetos do Python e as camadas visuais estruturadas no HTML pelo mecanismo de *templates* do Django. O sistema tratou com rigidez o caso **CT-01**, comprovando que qualquer índice pluviométrico acima de zero milímetros força a aplicação imediata do rótulo de risco severo, visando mitigar perdas de sustentação ou turbulências severas em voo.

No cenário **CT-02**, o limite operacional imposto pelas normativas desportivas foi respeitado na zona de ventos contínuos moderados. O **CT-03** comprovou a calibração adequada para a liberação da rampa sob condições atmosféricas plenamente seguras de inflagem de velames. Por fim, o caso **CT-04** validou a automação do sistema após a restauração completa do repositório original via Git (`git checkout`), provando que o *frontend* e o backend trabalham de maneira síncrona ao capturar e exibir dinamicamente a data e o clima atual de forma autônoma e em tempo real.